Q1 Use LIME to explain a prediction from a simple ML model.

```
!pip install lime
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.11/dist-packages (0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from lime) (1.15.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.11/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.11/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (11.2.1)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2.37.0)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2025.5.26)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (24.2)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lime) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (4.58.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (1.4.8)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from lime.lime_tabular import LimeTabularExplainer
```

```
# Loading the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target
feature_names = iris.feature_names
class_names = iris.target_names
```

```
print(feature_names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

```
print(class_names)
```

```
['setosa' 'versicolor' 'virginica']
```

```
# splitting the data into training and testing sets
# I am usING 80% for training and 20% for testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Training a Logistic Regression classifier
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression(max_iter=200)
```

```
# Picking one test sample to explain
sample_index = 5
x_sample = X_test[sample_index].reshape(1, -1)
true_label = y_test[sample_index]
predicted_label = model.predict(x_sample)[0]
predicted_proba = model.predict_proba(x_sample)[0]
```

```
print(f"Test sample index: {sample_index}")
```

```
Test sample index: 5
```

```
print("   Feature values for this sample:")
for fname, fval in zip(feature_names, x_sample[0]):
    print(f"    ~ {fname}: {fval:.2f}")
```

```
   Feature values for this sample:
    ~ sepal length (cm): 5.40
    ~ sepal width (cm): 3.40
    ~ petal length (cm): 1.50
    ~ petal width (cm): 0.40
```

```
print(f" True class label : {class_names[true_label]}")
```

```
 True class label : setosa
```

```
print(f" Model’s prediction  : {class_names[predicted_label]}")
```

```
 Model’s prediction  : setosa
```

```
print(f"Prediction probabilities: {dict(zip(class_names, np.round(predicted_proba, 3)))}\n")
```

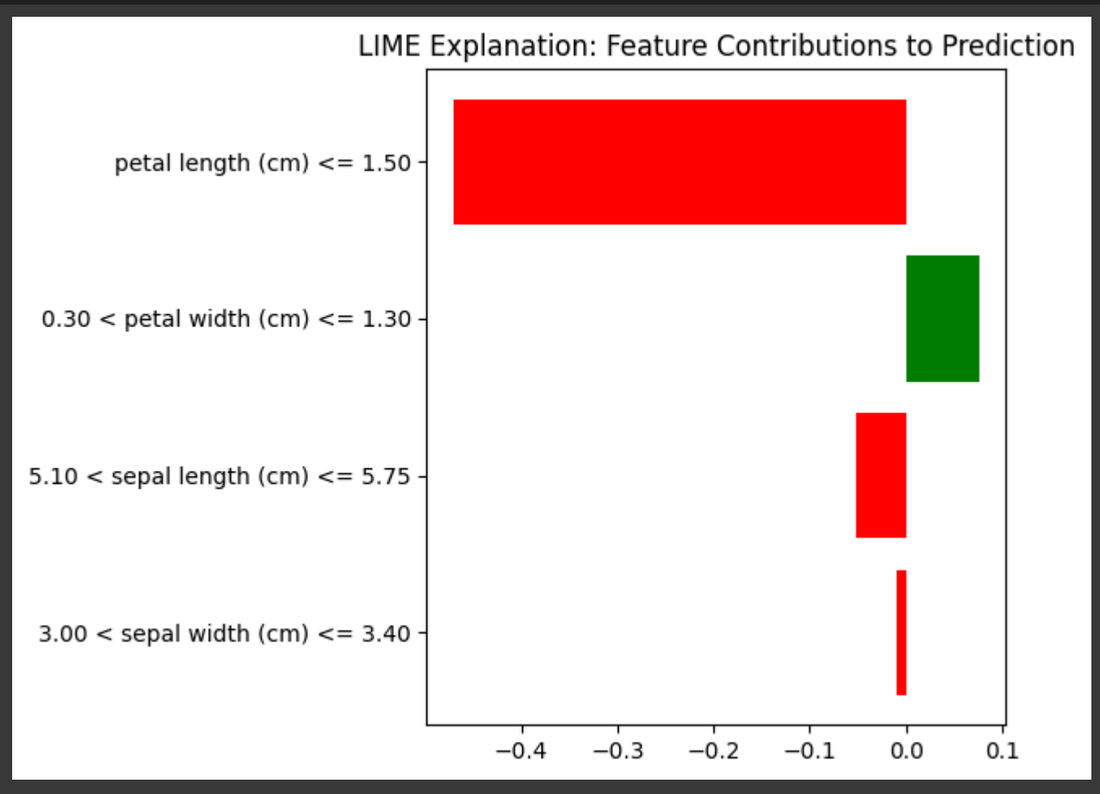
```
Prediction probabilities: {np.str_('setosa'): np.float64(0.956), np.str_('versicolor'): np.float64(0.044), np.str_('virginica'): np.float64(0.0)}
```

```
# Setting up LIME explainer on the training data

explainer = LimeTabularExplainer(
    X_train,
    feature_names=feature_names,
    class_names=class_names,
    discretize_continuous=True,   # LIME will bin continuous features for simplicity
    mode='classification'
)
```

```
# Explaining the prediction for our chosen sample
exp = explainer.explain_instance(
    x_sample[0],                # the raw feature values (this has no reshape)
    model.predict_proba,        # prediction function (it will return the probabilities)
    num_features=4              # show up to 4 features in explanation
)
```

```
# Displaying the LIME feature-contribution plot
fig = exp.as_pyplot_figure()
plt.title("LIME Explanation: Feature Contributions to Prediction")
plt.tight_layout()
plt.show()
```




```
# Printing the top 3 features that most influenced the prediction
top_features = exp.as_list()[:3]
```

```
print("->> Top 3 features influencing this prediction (feature: weight):")
for feat, weight in top_features:
    print(f"   ~       {feat}: {weight:.3f}")
```

```
->> Top 3 features influencing this prediction (feature: weight):
   ~       petal length (cm) <= 1.50: -0.471
   ~       0.30 < petal width (cm) <= 1.30: 0.077
   ~       5.10 < sepal length (cm) <= 5.75: -0.052
```

```
# final predicted class :
print(f"\n ->> Final predicted class according to LIME: {class_names[predicted_label]}")
```

```

 ->> Final predicted class according to LIME: setosa
```

Q2 Use LIME to interpret CNN predictions on image data.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from lime import lime_image
from skimage.segmentation import mark_boundaries
```

```
# Loading FashionMNIST dataset
transform = transforms.Compose([
    transforms.ToTensor(),                  # Converting to tensor
    transforms.Normalize((0.5,), (0.5,))     # Normalizing to [-1,1]
])
```

```
train_ds = torchvision.datasets.FashionMNIST(root='./data', train=True, download=True, transform=transform)
```

```
100%|██████████| 26.4M/26.4M [00:01<00:00, 16.1MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 275kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.06MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 12.3MB/s]
```

```
test_ds = torchvision.datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)
```

```
# Using a small subset for quick training
subset_indices = list(range(2000))  # first 2000 samples
train_subset = torch.utils.data.Subset(train_ds, subset_indices)
```

```
train_loader = torch.utils.data.DataLoader(train_subset, batch_size=64, shuffle=True)
```

```
test_loader = torch.utils.data.DataLoader(test_ds, batch_size=1, shuffle=False)
```

```
class_names = train_ds.classes
```

```
# Building a small CNN
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # Conv layer 1:
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        # Conv layer 2:
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        # Fully connected:
        self.fc1 = nn.Linear(32 * 7 * 7, 64)
        # Output layer:
        self.fc2 = nn.Linear(64, 10)
        self.relu = nn.ReLU()
```

```
    def forward(self, x):
        x = self.relu(self.conv1(x))
        x = self.pool(x)
        x = self.relu(self.conv2(x))
        x = self.pool(x)
        x = x.view(-1, 32 * 7 * 7)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = SimpleCNN().to(device)
```

```
# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
num_epochs = 3
```

```
for epoch in range(num_epochs):
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
    avg_loss = running_loss / len(train_loader.dataset)
    print(f"Epoch {epoch+1}/{num_epochs} - Loss: {avg_loss:.4f}")

print("Finished Training CNN on subset.")
```

```
Epoch 1/3 - Loss: 1.5203
Epoch 2/3 - Loss: 0.7908
Epoch 3/3 - Loss: 0.6458
Finished Training CNN on subset.
```

```
# Selecting one test image
# Using the first test sample for explanation
```

```
test_iter = iter(test_loader)
```

```
test_img, test_label = next(test_iter)
```

```
test_img = test_img.to(device)
```

```
test_label = test_label.item()
```

```
# Running the model to get a prediction
with torch.no_grad():
    logits = model(test_img)
    pred_prob = torch.softmax(logits, dim=1).cpu().numpy()[0]
    pred_class = np.argmax(pred_prob)
```

```
print(f"\nTrue label: {class_names[test_label]}")
```

```
True label: Pullover
```

```
print(f"Predicted label: {class_names[pred_class]}")
```

```
Predicted label: Pullover
```

```
# Applying LIME for image classification
# LIME expects a function that takes a numpy array and returns predictions
```

```
def predict_fn(images_np):
    model.eval()
    # Extracting first channel
    gray = images_np[:, :, :, 0]
    # Converting to tensor shape
    batch = torch.tensor(gray, dtype=torch.float32).unsqueeze(1).to(device)
    # Normalizing from [0..1] to [-1..1]
    batch = batch * 2 - 1
    with torch.no_grad():
        logits = model(batch)
        probs = torch.softmax(logits, dim=1)
    return probs.cpu().numpy()
```

```
# Creating LIME image explainer
explainer = lime_image.LimeImageExplainer()
```

```
# Convert selected test image to numpy format
test_img_np = test_img.squeeze().cpu().numpy()
```

```
test_img_np = ((test_img_np + 1) / 2) # undo normalization back to [0,1]
```

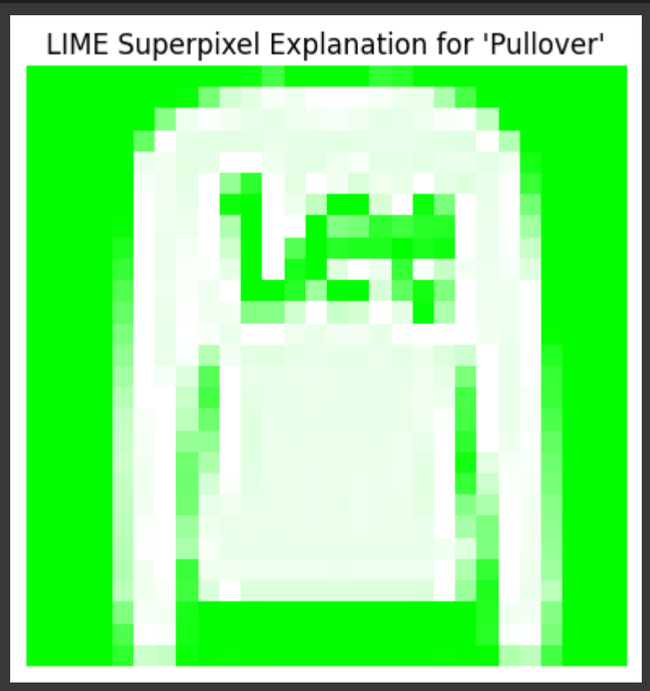
```
test_img_np = np.stack([test_img_np, test_img_np, test_img_np], axis=2)  # converting to 3 channels
```

```
# Generating explanation
explanation = explainer.explain_instance(
    test_img_np,
    classifier_fn=predict_fn,
    top_labels=1,
    hide_color=0,
    num_samples=1000
)
```

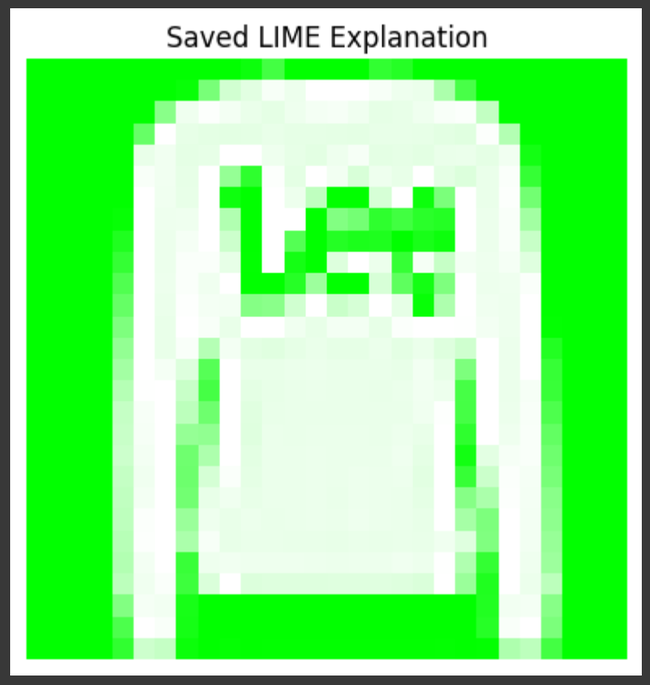
```
100% [                                    ] 1000/1000 [00:00<00:00, 2557.59it/s]
```

```
# Getting superpixel explanation for predicted class
temp, mask = explanation.get_image_and_mask(
    label=pred_class,
    positive_only=False,
    num_features=10,
    hide_rest=False
)
```

```
# Showing superpixel segmentation with explanation overlay
plt.imshow(mark_boundaries(temp, mask))
plt.title("LIME Superpixel Explanation for 'Pullover'")
plt.axis('off')
plt.show()
```




```
# Saving and showing the LIME visualization
output_path = "lime_fashionmnist_explanation.png"
plt.imsave(output_path, mark_boundaries(temp, mask))
print(f"LIME explanation saved to: {output_path}")
```

```
LIME explanation saved to: lime_fashionmnist_explanation.png
```

```
import matplotlib.image as mpimg

img = mpimg.imread('lime_fashionmnist_explanation.png')
plt.imshow(img)
plt.title("Saved LIME Explanation")
plt.axis('off')
plt.show()
```